Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Geçen dersin özeti: entropi ve artıklık
  - Bilgi doğrudan ölçülemediği için olasılıkla ölçülebilir hale getirilir: $I = \log_2 \frac{1}{P}$. Entropi $H = -\sum P \log_2 P$, sıkıştırılacak metinde bir sembolün ortalama kaç bitle kodlanabileceğini verir.
  - Artıklık (redundancy), seçilen yöntemin ortalama mesaj uzunluğu (S) ile entropi arasındaki farktır. Ne kadar küçükse yöntem o kadar idealdir.
- Move-To-Front (MTF) kodlama (ekrandan, 00:10:00)
  - Slayttaki tanım: bölgesel adaptif bir entropi kodlama yöntemidir; eldeki alfabe kullanılarak input stream'deki karakterlere kullanım sıklıklarına bağlı bir kod verilir; sıklıklar önceden hesaplanmaz; sıkıştırma performansını artırmak için asıl yöntemden önce kullanılan bir ön işlemdir.
  - İşleyiş: sıradaki sembolün alfabedeki indisi (0'dan başlayarak) yazılır, sonra sembol alfabenin en önüne taşınır, diğerleri kaydırılır.
  - Örnek (ekrandan, 00:20:00): input `a b c d d c b a m n o p p o n m`, alfabe `{abcdmnop}`. Kod dizisi `0 1 2 3 0 1 2 3 4 5 6 7 0 1 2 3`, alfabenin son hali `mnopabcd`. Kod dizisinin entropisi $H = 2{,}81$ bit/karakter.
  - Aynı input MTF'siz, yalnız alfabedeki indislerle kodlanırsa (ekrandan, 00:22:30) dizi `0 1 2 3 3 2 1 0 4 5 6 7 7 6 5 4` olur ve $H = 3$ bit çıkar.
  - Kod dizisinin kendisi sonra Huffman ya da aritmetik kodlama gibi istatistiksel bir yöntemle sıkıştırılır (ekrandan, 00:20:00).
  - MTF kelime bazında da yapılabilir (ekrandan, 00:30:00): `the boy on my right is the right boy`. Sözlük boş başlar; yeni kelime sözlüğün sonuna eklenip indisiyle birlikte yazılır (`0the 1boy 2on 3my 4right 5is`), daha önce görülmüş kelime için yalnız indis yazılır (`5 2 5`). Her adımda kelime sözlüğün başına taşınır.
  - Burrows-Wheeler Transform (BWT) de veriye uygulanan benzer bir ön işlem (dönüşüm) yöntemi olarak anıldı.
- Prefix kod (ekrandan, 00:50:00)
  - Olasılık tabanlı bütün yöntemlerin (Shannon-Fano, Huffman, aritmetik kodlama) ürettiği kodlar prefix özelliğine sahip olmalıdır; aksi halde kod tek anlamlı çözülemez.
  - Gerekli ve yeterli şart: bir sembolün kodu başka bir sembolün kodunun başlangıcı (ön eki) olmamalıdır.
  - Örnek: A=1, B=11 seçilirse `baba` kodlaması altı tane 1'e döner ve hem `AAAAAA` hem `BBB` diye açılabilir.
  - Code1 (`1, 01, 010, 001`) prefix değildir: $a_3a_2 \rightarrow$ `01001`, bu dizi `01|001` = a_2a_4 diye açılır. Code2 (`1, 01, 000, 001`) prefix'tir ve `000|01` doğru açılır.
  - İnteger sayıların düz binary gösterimi prefix değildir (1 ile başlayan birçok kod vardır).
- Değişken uzunluklu kodlar (ekrandan, 00:40:00 ve 00:50:00)
  - 4 eşit olasılıklı sembol: $H = 2$ bit, 2 bitlik sabit kodla $S = 2$, $R = 0$.
  - Olasılıklar 0,49 / 0,25 / 0,25 / 0,01 olunca $H = 1{,}57$ bit; 2 bitlik sabit kodla $R = 0{,}43$.
  - 20 sembollük girdi (9 a_1, 5 a_2, 5 a_3, 1 a_4) 1, 2, 3, 3 bitlik kodlarla 37 bit tutar (8 bitle 160 bit). $S = 37/20 = 1{,}85$, slayttaki $R = 0{,}28$.
- Hazır (statik) prefix kod örnekleri
  - Logaritmik prefix kod (ekrandan, 00:56:40): $i \geq 1$ için i'nin binary değerinin başına $\lfloor \log_2 i \rfloor$ adet 0 eklenir. `1, 010, 011, 00100, ..., 0001000, ..., 000010000` (16 için

9 bit).
  - Unary kod (ekrandan, 01:20:00): n için $(n-1)$ adet 1 ve bir 0 (ya da $(n-1)$ adet 0 ve bir 1). n 0'dan başlıyorsa n adet 1 ve bir 0.
  - Start-step-stop kodu: $a = \text{start} + n \cdot \text{step}$; kod, n'in unary kodu ve arkasından a bitlik binary sayıdır; $a = \text{stop}$ olunca unary kısmın sonundaki 0 düşürülür. (3,2,9) tripleti $8 + 32 + 128 + 512 = 680$ kod üretir.
  - Toplam kod sayısı: $\frac{2^{\text{stop}+\text{step}} - 2^{\text{start}}}{2^{\text{step}} - 1}$.
  - C_1, C_2, C_3, C_4 kodları (ekrandan, 01:25:20): $B(n)$ binary gösterim, $|B(n)|$ dijit sayısı, $\bar{B}(n)$ en soldaki bit atılınca kalan bitler.
  - Golomb kodu (ekrandan, 01:50:00): semboller olasılığa göre büyükten küçüğe sıralanıp $x = 0, 1, 2, \ldots$ değerleri alır; 2'nin kuvveti bir M seçilir; $x_M = \lfloor x/M \rfloor$, $x_L = x \bmod M$; kod = unary(x_M) + binary(x_L).
- İstatistiksel yöntemlerin sınıfları
  - Statik: kodlar önceden hazırdır (unary, C_1-C_4, Golomb gibi), semboller olasılıklarına göre bu kodlara atanır.
  - Yarı statik (semi-adaptif): önce metin okunup olasılıklar çıkarılır, sonra kod atanır. Metne göre kod üretildiği için daha iyi sıkıştırır.
  - Adaptif: bir sembolün kodu metin boyunca değişebilir (3 bitten 5 bite, 5 bitten 4 bite). Daha karmaşık ve yavaş, ama sıkıştırma performansı daha yüksektir.
- Shannon-Fano kodlama (ekrandan, 02:10:00)
  - Olasılık tabanlı, yarı statik, değişken uzunluklu, prefix kod üreten yöntem.
  - Semboller olasılığa göre azalan sırada dizilir; küme toplam olasılıkları birbirine en yakın iki parçaya bölünür; bir parçaya 1, diğerine 0 verilir; her parça tek eleman kalana kadar aynı biçimde bölünür. Bölmeler bir ağacın seviyelerini oluşturur.
  - Olasılıklar 2'nin negatif kuvvetleri $(1/2, 1/4, 1/8, \ldots)$ olduğunda en iyi sonucu verir.
- Huffman kodlamaya giriş (ekrandan, 02:30:00 ve 02:40:00)
  - Tek başına ya da başka bir yöntemin içinde ara adım olarak kullanılır (JPEG, ZIP, WinRAR).
  - En küçük iki olasılık toplanır, toplam sıralamayı bozmayacak yere yerleştirilir; tek değer kalana kadar sürer. Kodlar kökten yapraklara doğru okunur.
  - Toplam, var olan eşit bir olasılığın önüne ya da arkasına konabilir; iki yol farklı kodlar verir ama ortalama uzunluk aynı kalır.

Hocanın Özellikle Vurguladığı Kısımlar

- Prefix özelliği
  - Olasılık kullanan bütün sıkıştırma algoritmaları prefix kod üretir; prefix olmayan kod geri açılamaz. Bir prefix kod kümesi kurulurken ön ek olacak kodlar (ör. tek başına `0`, `00`) kullanılmaz.
- Olasılık ve bit sayısı ilişkisi
  - Olasılığı yüksek sembol az bilgi taşır ve az bitle, olasılığı düşük sembol çok bilgi taşır ve çok bitle kodlanmalıdır. Çok geçen sembole 8 yerine 1 bit vermenin kazancı, nadir geçen sembole 14 bit vermenin kaybından büyüktür.
- Kod seçimi veriye bağlıdır
  - C_1-C_4 kodları her n için aynı uzunlukta değildir (ör. $n = 4$: C_1, C_2, C_3 5 bit, C_4 6 bit; $n = 15$: C_3 8 bit, C_4 7 bit). Hangisinin seçileceğine olasılık dağılımına bakılarak karar verilir.
- Entropi değişmez, ortalama uzunluk değişir
  - Shannon-Fano'da bölme noktası ya da Huffman'da toplamın yerleştirildiği yer değişince kodlar ve bazen ortalama uzunluk değişir; entropi aynı kalır. Amaç artıklığı en küçük yapan yöntemi seçmektir.
- Huffman ve Shannon-Fano karşılaştırması
  - Huffman, Shannon-Fano'dan daha iyi sıkıştırır (artıklığı daha küçüktür). Olasılıklar 2'nin kuvvetleri cinsindeyse Shannon-Fano zaten en iyi sonucu verir ve daha basittir. Yöntem seçimi amaca bağlıdır: yer mi zaman mı, sıkıştırma mı açma mı hızlı olmalı.
- Huffman'ın kullanım alanları
  - Hoca Huffman'ın JPEG, ZIP ve WinRAR içinde ara adım olarak kullanıldığını defalarca söylediğini ve dönem sonunda bunun bilinmesini beklediğini belirtti.

Kısa Tekrar Notları

- MTF: sembolün indisini yaz, sembolü alfabenin başına taşı. Ön işlemdir; asıl sıkıştırma kod dizisine uygulanır.
- Örnekte MTF ile $H = 2{,}81$, MTF'siz $H = 3$ bit. Alfabe büyüdükçe (ör. kelime bazında) MTF'nin faydası artar.
- Prefix şartı: hiçbir kod başka bir kodun ön eki değildir.
- Logaritmik prefix kod: $\lfloor \log_2 i \rfloor$ sıfır + binary(i), uzunluk $2\lfloor \log_2 i \rfloor + 1$.
- Unary: $n-1$ tane 1 + 0. Start-step-stop: $a = \text{start} + n \cdot \text{step}$, son grupta (a = stop) sondaki 0 düşer. (3,2,9) → 680 kod.
- C_1 = unary($|B(n)|$) + $\bar{B}(n)$; uzunluk $2\lfloor \log_2 n \rfloor + 1$.
- C_2 = C_1'in iki parçasının bitleri sırayla birer birer araya karıştırılmış hali.
- C_3 = $C_2(|B(n)|)$ + $\bar{B}(n)$.
- C_4 = binary(n), soluna $|B| - 1$'in binary'si, iki dijit kalana kadar tekrar, en sağa 0.
- Golomb: $x_M = \lfloor x/M \rfloor$, $x_L = x \bmod M$, kod = unary(x_M) + binary(x_L).
- Statik, yarı statik (semi-adaptif), adaptif yöntemler.
- Shannon-Fano: azalan sırala, olasılıkça en dengeli ikiye böl, üste 1 alta 0, tek eleman kalana kadar sür.
- Huffman: en küçük ikiyi topla, sıralamayı bozmadan yerleştir, kökten yaprağa oku.

Detaylı Açıklamalar

- MTF örneğinin adımları
  - Alfabe `abcdmnop`. `a` 0. indiste → 0. `b` 1. indiste → 1, alfabe `bacdmnop`. `c` → 2, `cbadmnop`. `d` → 3, `dcbamnop`. Tekrar `d` → 0. `c` → 1, `b` → 2, `a` → 3, `m` → 4, `n` → 5, `o` → 6, `p` → 7, `p` → 0, `o` → 1, `n` → 2, `m` → 3. Son alfabe `mnopabcd` (ekrandan, 00:20:00).
  - Kod dizisinde 0, 1, 2, 3 üçer kez, 4-7 birer kez geçer: $P(0) = 3/16$ gibi. $H = 4 \cdot \frac{3}{16} \log_2 \frac{16}{3} + 4 \cdot \frac{1}{16} \cdot 4 \approx 2{,}81$ bit (python ile doğrulandı). Sabit 8 bit yerine ortalama 2,81 bit.
  - MTF'siz dizide her indis iki kez geçer, 8 eşit olasılık: $H = 3$ bit.
  - Hoca, alfabe küçük olduğu için farkın az göründüğünü, kelime kodlamada (on binlerce kelime) MTF'nin çok daha yararlı olacağını söyledi.
  - Kod çözmede slayt "input tersten okunur ve alfabenin ilk hali oluşturulur" diyor (ekrandan, 00:20:00). Hoca alfabenin son hali `mnopabcd` ile başlayıp kodları uyguladı, çıkan diziyi tersten okuyarak özgün inputu buldu.
  - **Ders sonrası not:** Hocanın gösterdiği yol (son alfabeden başlayıp kodları baştan sona uygulamak ve sonucu ters çevirmek) bu örnekte input simetrik olduğu için doğru sonuç veriyor; genel bir girdi için çalışmaz (python ile denendi). Genel kod çözme iki biçimde yapılır: (1) alfabenin ilk haliyle başlayıp kodlar baştan sona uygulanır: indisteki sembol yazılır ve başa taşınır; (2) son alfabeyle başlayıp kodlar sondan başa işlenir: baştaki sembol yazılır ve kodun gösterdiği indise geri konur, çıkan dizi ters çevrilir. Kelime bazlı örnekte hoca boş sözlükle baştan sona çözdü; bu (1) numaralı yoldur ve doğrudur.
- Kelime bazlı MTF çözümü
  - Sözlük boş başlar. `0the`: 0. indise `the` konur, yazılır. `1boy`, `2on`, `3my`, `4right`, `5is` aynı şekilde; her kelime yazıldıktan sonra sözlüğün başına taşınır. Sonra yalnız sayılar gelir: `5` → sözlükte 5. indis `the`; `2` → `right`; `5` → `boy`. Hoca, sözlükte olmayan bir kelime (ör. `girl`) gelseydi `6girl` yazılacağını söyledi.
- Prefix kod örneği (ekrandan, 00:50:00)
  - Code1: a_1=1, a_2=01, a_3=010, a_4=001. Code2: a_1=1, a_2=01, a_3=000, a_4=001.
  - a_3a_2 Code1 ile `010 01`; çözücü `01` görünce a_2 der, kalan `001` a_4 olur: yanlış. Code2 ile `000 01` → a_3, a_2: doğru.
  - Code1'de a_2'nin kodu `01`, a_3'ün kodu `010`'ın ön ekidir; prefix değildir.
  - **Ders sonrası not:** Slayttaki tabloda a_4'ün olasılığı 0,02 yazıyor (toplam 1,01 oluyor); önceki slaytta ve $H = 1{,}57$ hesabında 0,01 kullanılmış. $R = |1{,}57 - 1{,}85| = 0{,}28$. Girdi dizisindeki sayımlardan (9/20, 5/20, 5/20, 1/20) entropi hesaplanırsa $H \approx 1{,}73$ ve $R \approx 0{,}11$ çıkar (python ile hesaplandı).
- Logaritmik prefix kod (ekrandan, 00:56:40)
  - Hoca önce taban ($\lfloor 5{,}1 \rfloor = 5$) ve tavan ($\lceil 5{,}1 \rceil = 6$) fonksiyonlarını sayı doğrusu üze-

rinde hatırlattı.
  - $i = 1$: $\log_2 1 = 0$, kod `1`. $i = 2, 3$: bir 0, `010`, `011`. $i = 4..7$: iki 0, `00100` ... `00111`. $i = 8$: üç 0, `0001000`. $i = 16$: dört 0, `000010000` (9 bit).
  - **Ders sonrası not:** Hoca 64 kod üretilirse en düşük olasılıklı sembolün 9 bitle kodlanacağını söyledi. Formüle göre $i = 64$ için $\lfloor \log_2 64 \rfloor = 6$ sıfır ve 7 bitlik binary, toplam 13 bit olur; 9 bit $i = 16$ içindir.
- Start-step-stop örneği (3,2,9) (ekrandan, 01:20:00)

```
n  a=3+2n  kod biçimi          kod sayısı   aralık
0  3       0xxx                2^3 = 8      0-7
1  5       10xxxxx             2^5 = 32     8-39
2  7       110xxxxxxx          2^7 = 128    40-167
3  9       111xxxxxxxxx        2^9 = 512    168-679
```

- $n = 3$'te $a = 9 =$ stop olduğu için unary kısımdaki 0 düşer (`1110` yerine `111`). Arkasından başka grup gelmediği için prefix özelliği bozulmaz; 512 kodun her birinde bir bit kazanılır.
- Formülle: $\frac{2^{11}-2^3}{2^2-1} = \frac{2040}{3} = 680$.
- C_1-C_4 örnekleri (ekrandan, 01:25:20, 01:30:00, 01:34:50, 01:40:00)
  - $C_1(16)$: $B = 10000$, $|B| = 5$, unary(5) = `11110` (ya da `00001`), $\bar{B} = 0000 \rightarrow$ `11110|0000` (9 bit).
  - $C_1(6)$: $B = 110$, $|B| = 3 \rightarrow$ `110|10`.
  - $C_2(16)$: `11110` ile `0000` bitleri sırayla karıştırılır → `101010100`.
  - $C_3(15)$: $|B(15)| = 4$, $C_2(4) = 10100$, $\bar{B}(15) = 111 \rightarrow$ `10100|111`.
  - $C_4(16)$: `10000`, $5 - 1 = 4 \rightarrow$ `100|10000`, $3 - 1 = 2 \rightarrow$ `10|100|10000`, sağa 0 → `10|100|10000|0`.
  - $C_4(5)$: `101`, $3 - 1 = 2 \rightarrow$ `10|101`, sağa 0 → `10|101|0`.
  - **Ders sonrası not:** C_3 uzunluğu derste karışık hesaplandı. Tanıma göre uzunluk $|C_2(|B(n)|)| + |B(n)| - 1$'dir: $n = 16$ için $5 + 4 = 9$, $n = 15$ için $5 + 3 = 8$ bit (python ile doğrulandı).
- Golomb örneği, $M = 4$ (ekrandan, 01:50:00)

```
x   xM  xL  unary(xM)  binary(xL)  Golomb
0   0   0   0          00          000
1   0   1   0          01          001
2   0   2   0          10          010
3   0   3   0          11          011
4   1   0   10         00          1000
5   1   1   10         01          1001
6   1   2   10         10          1010
7   1   3   10         11          1011
8   2   0   110        00          11000
```

- x_M 0'dan başladığı için unary kodu x_M adet 1 ve bir 0'dır. Orijinal değere $x = x_M \cdot M + x_L$ ile dönülür.
- Shannon-Fano örneği 1 (ekrandan, 02:20:00)
  - Olasılıklar: 0,25; 0,20; 0,15; 0,15; 0,10; 0,10; 0,05. İlk bölme 0,45 / 0,55 (ikinci sembolden sonra). Kodlar: `11, 10, 011, 010, 001, 0001, 0000`.
  - $H = 2{,}67$ bit, $S = 0{,}25 \cdot 2 + 0{,}20 \cdot 2 + 2 \cdot 0{,}15 \cdot 3 + 0{,}10 \cdot 3 + 0{,}10 \cdot 4 + 0{,}05 \cdot 4 = 2{,}7$ bit, $R = 0{,}03$ (python ile doğrulandı).
- Shannon-Fano örneği 2 (ekrandan, 02:26:30)
  - Olasılıklar: altı tane 0,15 ve bir 0,10. 60/40 yerine 45/55 bölmesi seçilir (fark 20 yerine 10). Üst grup 0,30 / 0,15 bölünür (büyük olasılık üstte kalacak kuralı). Kodlar: `111, 110, 10, 011, 010, 001, 000`.
  - Aynı olasılıklı (0,15) semboller 2 ya da 3 bitle kodlanabilir; bölme yeri kodları ve ortalama uzunluğu değiştirir, entropiyi değiştirmez.
  - Derste H ve S hesaplanmadı. Python ile: $H \approx 2{,}80$ bit, $S = 2{,}85$ bit, $R \approx 0{,}05$.
- Huffman örneği (ekrandan, 02:30:00 ve 02:40:00)
  - Olasılıklar: $a_1 = 0{,}4$, $a_2 = 0{,}2$, $a_3 = 0{,}2$, $a_4 = 0{,}1$, $a_5 = 0{,}1$.
  - Birinci yol: `0, 10, 111, 1101, 1100`; $S = 0{,}4 \cdot 1 + 0{,}2 \cdot 2 + 0{,}2 \cdot 3 + 0{,}1 \cdot 4 + 0{,}1 \cdot 4 = 2{,}2$ bit.
  - İkinci yol: `11, 01, 00, 101, 100`; $S = 0{,}4 \cdot 2 + 0{,}2 \cdot 2 + 0{,}2 \cdot 2 + 0{,}1 \cdot 3 + 0{,}1 \cdot 3 = 2{,}2$

bit.
  - Entropi derste hesaplanmadı; python ile $H \approx 2{,}12$ bit, $R \approx 0{,}08$.
  - Hoca iki yol arasında ne zaman seçim yapılması gerektiğini varyans konusuyla birlikte sonraki derste anlatacağını söyledi; ardından dinamik Huffman, canonical Huffman ve aritmetik kodlamaya geçilecek.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.